```
# ================================================
# Roll no : 60       PRN: 0124UITM1060
# Name : Kshitij Shinde
# Dept. : Information Technology (Third Year)
# ================================================
# Assignment 1:
# Data Cleaning and Visualization for Retail Sales
# ================================================

# Import Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

# ------------------------------------------
# Load Dataset
# ------------------------------------------

df = pd.read_csv("retail_sales.csv")

print("Original Dataset")
```

```
print(df.head())

# ---------------------------------------------
# Check Missing Values
# ---------------------------------------------

print("\nMissing Values")
print(df.isnull().sum())

# ---------------------------------------------
# Replace Missing Values with Mean
# ---------------------------------------------

numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    df[col].fillna(df[col].mean(), inplace=True)

print("\nDataset After Removing Missing Values")
print(df.head())

# ---------------------------------------------
# Variance
```

```
# ---------------------------------------------

print("\nVariance")
print(df[numeric_columns].var())

# ---------------------------------------------
# Standard Deviation
# ---------------------------------------------

print("\nStandard Deviation")
print(df[numeric_columns].std())

# ---------------------------------------------
# Covariance Matrix
# ---------------------------------------------

print("\nCovariance Matrix")
print(df[numeric_columns].cov())

# ---------------------------------------------
# Correlation Matrix
# ---------------------------------------------
```

```
print("\nCorrelation Matrix")
print(df[numeric_columns].corr())

# ---------------------------------------------
# Normalization (Min-Max)
# Formula = (X - Min) / (Max - Min)
# ---------------------------------------------

normalized_df = df.copy()

for col in numeric_columns:
    minimum = df[col].min()
    maximum = df[col].max()

    normalized_df[col] = (df[col] - minimum) / (maximum - minimum)

print("\nNormalized Data")
print(normalized_df.head())

# ---------------------------------------------
# Standardization (Z-Score)
# Formula = (X - Mean) / Standard Deviation
# ---------------------------------------------
```

```
standardized_df = df.copy()

for col in numeric_columns:
    mean = df[col].mean()
    std = df[col].std()

    standardized_df[col] = (df[col] - mean) / std

print("\nStandardized Data")
print(standardized_df.head())

# ------------------------------------------
# Discretization
# ------------------------------------------

if "Sales" in df.columns:

    df["Sales_Category"] = pd.cut(
        df["Sales"],
        bins=3,
        labels=["Low", "Medium", "High"]
    )
```

```
    print("\nDiscretized Sales")
    print(df[["Sales", "Sales_Category"]].head())

# ------------------------------------------
# Histogram
# ------------------------------------------

if "Sales" in df.columns:

    plt.figure(figsize=(6,4))
    plt.hist(df["Sales"], bins=10, edgecolor="black")
    plt.title("Histogram of Sales")
    plt.xlabel("Sales")
    plt.ylabel("Frequency")
    plt.show()

# ------------------------------------------
# Boxplot
# ------------------------------------------

if "Sales" in df.columns:
```

```
    plt.figure(figsize=(6,4))
    plt.boxplot(df["Sales"])
    plt.title("Boxplot of Sales")
    plt.show()

# ---------------------------------------------
# Detect Outliers using IQR
# ---------------------------------------------

if "Sales" in df.columns:

    Q1 = df["Sales"].quantile(0.25)
    Q3 = df["Sales"].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    cleaned_df = df[(df["Sales"] >= lower) & (df["Sales"] <= upper)]

    print("\nOriginal Rows :", len(df))
    print("Rows After Removing Outliers :", len(cleaned_df))
```

```
# ------------------------------------------
# Scatter Plot
# ------------------------------------------

if "Sales" in df.columns and "Profit" in df.columns:

    plt.figure(figsize=(6,4))
    plt.scatter(df["Sales"], df["Profit"])
    plt.title("Sales vs Profit")
    plt.xlabel("Sales")
    plt.ylabel("Profit")
    plt.show()

# ------------------------------------------
# Quantile (Q-Q) Plot
# ------------------------------------------

if "Sales" in df.columns:

    plt.figure(figsize=(6,4))
    stats.probplot(df["Sales"], dist="norm", plot=plt)
    plt.title("Q-Q Plot")
```

```
    plt.show()


print("\nData Preprocessing Completed Successfully.")
```